

dashboard

dashboard

概述

本文档介绍如何在 Grafana 中使用 DOG Stack 的可观测数据。

DOG Stack 使用 Doris 作为统一的存储后端。Doris 兼容 MySQL 协议，在 Grafana 中通过 MySQL 数据源即可查询所有 Logs、Traces 和 Metrics 数据。数据链路如下：

应用 / 基础设施 → OpenTelemetry Collector → Doris Exporter → Doris → Grafana (MySQL DataSource)

本文档包含以下内容：

- 导入预置 Dashboard：使用我们提供的现成 Dashboard。
- 理解数据模型：了解 Doris 中 OTel 数据的存储方式。
- 创建自定义 Dashboard：从零开始构建面板、编写查询、配置变量。
- 参考：表结构和语法速查。

准备工作

在开始之前，确保满足以下条件：

- 已部署 DOG Stack，Grafana 和 Doris 均正常运行。
- 已在 Grafana 中配置 MySQL 类型的数据源，连接到 Doris（DOG Stack 启动的 Grafana 已经预配置了 Doris 数据源）。

如果尚未配置数据源，按以下步骤操作：

1. 在 Grafana 左侧菜单中，点击 **Connections > Data sources > Add data source**。
2. 选择 **MySQL**。
3. 填写以下连接信息：
 - **Host**: :9030

- **Database:** otel
- **User:** root (或你配置的用户)
- **Password:** 留空 (如未设置密码)

4. 点击 **Save & test**, 确认连接成功。

图片说明: 配置 MySQL 数据源

导入预置 Dashboard

我们提供了 4 个 预置 Dashboard, 覆盖常见的可观测场景:

- Dashboard: Host Metrics; 文件名: host_metrics_dashboard.json; 监控内容: CPU、内存、磁盘、网络、系统负载
- Dashboard: JVM Monitoring; 文件名: jvm_metrics_dashboard.json; 监控内容: 堆内存、GC、线程、CPU 利用率
- Dashboard: K8s Observability; 文件名: k8s_kubelet_dashboard.json; 监控内容: Pod / Node / Namespace 资源使用
- Dashboard: Nginx Logs; 文件名: nginx_logs_dashboard.json; 监控内容: 请求量、状态码、Top URL、错误日志
- Dashboard: PostgreSQL Metrics; 文件名: postgresql-metrics-dashboard.json; 监控内容: 连接数、事务、数据库大小、Checkpoint、BGWriter

要导入预置 Dashboard:

1. 在 Grafana 左侧菜单中, 点击 **Dashboards > New > Import**。
2. 点击 **Upload dashboard JSON file**, 选择 grafana-dashboard/ 目录下的 JSON 文件。
3. 在导入页面中, 将数据源选择为你配置的 Doris (MySQL 类型)。
4. 点击 **Import**。
5. 对其余 JSON 文件重复以上步骤。

导入完成后, Dashboard 顶部的变量选择器 (如 Service、Namespace 等) 会自动从 Doris 查询可选值。

各 Dashboard 面板详情

Host Metrics Dashboard

- Overview: CPU 使用率、内存使用率、1 分钟负载、根磁盘使用率
- CPU: 使用率趋势、按 Mode 分类的使用率
- Memory: 内存详情 (Total / Available / Free / Cached / Buffers)、使用率趋势
- System Load: 1 / 5 / 15 分钟负载、打开的文件描述符数
- Disk: 磁盘空间使用表格、读写吞吐量
- Network: 收发流量、错误与丢包

图片说明: Host Metrics Dashboard

JVM Monitoring Dashboard

- Overview: 堆使用率、Full GC 次数、CPU 使用率、线程数
- Memory: 堆 / 非堆内存趋势、Old Gen / Eden / Survivor 各池详情
- GC: 耗时 (Young / Old 分类)、次数
- Thread & CPU: 线程数趋势、CPU 利用率

图片说明: JVM Monitoring Dashboard

K8s Observability Dashboard

- Pods: CPU (毫核)、内存、Limit 利用率、状态表格 (含 Restart 次数、运行时长)
- Nodes: CPU / 内存趋势、状态表格
- Namespaces: CPU / 内存汇总、状态表格

图片说明: K8s Observability Dashboard

Nginx Logs Dashboard

- Overview: 请求数、错误数、5xx / 4xx 统计、状态码饼图
- Trends: 请求量 / 错误日志趋势
- Analysis: Top URL、Top IP、HTTP 方法分布、Top 错误信息
- Recent Logs: Access / Error 日志明细

图片说明: Nginx Logs Dashboard

PostgreSQL Metrics Dashboard

- Connections：活跃连接数（按数据库拆分）、最大连接数
- Transactions：事务提交速率、回滚次数
- Storage：数据库大小（按数据库拆分）、数据库数量、表数量
- BGWriter：Checkpoint 次数（scheduled / requested）、Buffer 写入次数、BGWriter 耗时

图片说明：PostgreSQL Metrics Dashboard

理解数据模型

创建自定义 Dashboard 之前，你需要了解 Doris 中 OTel 数据的存储方式。

数据表

与 Prometheus 不同，在 Doris 中你不能只通过 metric name 来查询，你还需要知道查哪张表。OTel 数据按信号类型和指标类型存储在不同的表中：

- 表名：otel_metrics_gauge；存储内容：瞬时值指标；值字段：value；查询方式：直接聚合，如 AVG(value)
- 表名：otel_metrics_sum；存储内容：累计计数器；值字段：value（单调递增）；查询方式：用 LAG() 计算速率
- 表名：otel_metrics_histogram；存储内容：直方图；值字段：count、sum、bucket_counts；查询方式：用 LAG() 计算增量
- 表名：otel_logs；存储内容：日志；值字段：body、severity_text；查询方式：COUNT 聚合或明细查询
- 表名：otel_traces；存储内容：链路追踪；值字段：duration、span_name；查询方式：按耗时排序或聚合

要确定一个指标存在哪张表中，运行以下查询：

```
SELECT 'gauge' AS type, metric_name FROM otel.otel_metrics_gauge WHERE
metric_name = '你的指标名' LIMIT 1
UNION ALL
SELECT 'sum', metric_name FROM otel.otel_metrics_sum WHERE metric_name = '你的指标名' LIMIT 1
UNION ALL
SELECT 'histogram', metric_name FROM otel.otel_metrics_histogram WHERE
metric_name = '你的指标名' LIMIT 1;
```

要浏览所有可用指标，对每张表运行以下查询：

SELECT DISTINCT metric_name **FROM** otel.otel_metrics_gauge **ORDER BY** metric_name;

属性字段

在 Prometheus 中，label 统一用 {key="value"} 访问。在 Doris 中，属性分散在多个 variant (JSON) 类型字段中，使用中括号语法访问：

- 表：metrics 表；字段名：attributes；存储内容：指标维度（对应 Prometheus label）；示例：attributes['mode']
- 表：metrics 表；字段名：resource_attributes；存储内容：资源信息（K8s pod/node 等）；示例：resource_attributes['k8s.pod.name']
- 表：logs 表；字段名：log_attributes；存储内容：日志字段；示例：log_attributes['status']
- 表：traces 表；字段名：span_attributes；存储内容：Span 字段；示例：span_attributes['http.method']

Note: service_name 和 service_instance_id 是顶层列，不在 attributes 内。直接使用 WHERE service_name = '...'。

在 SELECT、GROUP BY、PARTITION BY 或使用 LIKE 操作时，需要用 CAST 做类型转换：

-- 在 SELECT 中取值

CAST(attributes['device'] AS VARCHAR) AS device

-- 在 WHERE 中做模式匹配

CAST(attributes['device'] AS VARCHAR) NOT LIKE 'veth%'

-- 在 WHERE 中做数值比较

CAST(log_attributes['status'] AS INT) >= 500

-- 在 GROUP BY 中使用

GROUP BY CAST(resource_attributes['k8s.pod.name'] AS VARCHAR)

在 WHERE 中做简单等值比较时，通常不需要 CAST：

WHERE attributes['mode'] = 'idle'

Caution: 使用错误的属性字段名（例如在 metrics 表上用 log_attributes）不会报错，只会返回 NULL，导致查询结果为空。

Counter 指标与速率计算

Gauge 指标的 value 直接代表当前值，可以直接使用 AVG 或 MAX 聚合。

Counter/Sum 指标的 value 是单调递增的累计值（如 CPU 总秒数、网络总字节数）。要获取速率，需要计算相邻数据点的差值。Prometheus 的 rate() 在 Doris 中用 LAG() 窗口函数实现。

以下是通用的 rate 计算模板：

```
SELECT
  t.timestamp AS time,
  t.AS metric,
  CASE
    WHEN UNIX_TIMESTAMP(t.timestamp) > UNIX_TIMESTAMP(t.prev_ts)
      AND t.value >= t.prev_value
    THEN (t.value - t.prev_value) / (UNIX_TIMESTAMP(t.timestamp) -
UNIX_TIMESTAMP(t.prev_ts))
    ELSE NULL
  END AS value
FROM (
  SELECT timestamp, value, ,
    LAG(value) OVER (PARTITION BY ORDER BY timestamp) AS prev_value,
    LAG(timestamp) OVER (PARTITION BY ORDER BY timestamp) AS prev_ts
  FROM otel.otel_metrics_sum
  WHERE metric_name = "
    AND $__timeFilter(timestamp)
) t
WHERE t.prev_ts IS NOT NULL
ORDER BY time
```

使用这个模板时，替换以下占位符：

- ``：用于拆分时间线的字段，如 CAST(attributes['device'] AS VARCHAR)。
- ``：metric name，如 node_network_receive_bytes_total。

模板中包含三层安全保护：

- 条件：WHERE t.prev_ts IS NOT NULL；作用：过滤每个分区的第一行（LAG 返回 NULL）
- 条件：UNIX_TIMESTAMP(t.timestamp) > UNIX_TIMESTAMP(t.prev_ts)；作用：防止时间戳重复导致除零

- 条件：t.value >= t.prev_value；作用：防止 Counter 重置产生负数

PARTITION BY 的选择直接决定 rate 计算是否正确。选错会导致不同维度的数据交叉计算。以下是预置 Dashboard 中的用法：

- 场景：CPU 使用率（多核汇总）；PARTITION BY：service_instance_id, CAST(attributes['cpu'] AS VARCHAR)
- 场景：磁盘 I/O；PARTITION BY：CAST(attributes['device'] AS VARCHAR)
- 场景：网络流量；PARTITION BY：CAST(attributes['device'] AS VARCHAR)
- 场景：GC 耗时；PARTITION BY：CAST(attributes['jvm.gc.name'] AS VARCHAR)

创建自定义 Dashboard

本节通过实际操作，演示从零创建一个包含多种面板和变量的 Dashboard。

创建 Dashboard 和 Time Series 面板

本节创建一个 Time Series 面板，展示 K8s Pod 的 CPU 使用趋势。这个示例使用 Gauge 类型指标，构建过程展示了编写 SQL 的完整思路。

1. 在 Grafana 左侧菜单中，点击 **Dashboards > New > New dashboard**。
2. 点击 **Add visualization**。
3. 在数据源下拉框中，选择你配置的 MySQL（Doris）数据源。
4. 在查询编辑区右上角，点击切换到 **Code** 模式。

接下来编写 SQL 查询。以下逐步构建完整的查询语句。

编写基础查询。 目标是监控 Pod CPU，对应的指标名是 k8s.pod.cpu.usage。这是一个 Gauge 类型指标，存在 otel_metrics_gauge 表中，value 可以直接聚合：

```
SELECT timestamp, value
FROM otel.otel_metrics_gauge
WHERE metric_name = 'k8s.pod.cpu.usage'
```

添加时间过滤。 Grafana 的 MySQL 数据源提供了两种时间过滤方式：

```
-- 方式 A: 使用 $__timeFilter 宏 (推荐)
WHERE metric_name = 'k8s.pod.cpu.usage'
AND $__timeFilter(timestamp)
```

```
-- 方式 B: 使用 $__from 和 $__to (在子查询或 JOIN 中更灵活)
```

```
WHERE metric_name = 'k8s.pod.cpu.usage'
AND timestamp >= FROM_UNIXTIME($__from/1000)
AND timestamp = FROM_UNIXTIME($__from/1000)
AND timestamp = FROM_UNIXTIME($__from/1000)
AND timestamp **Unit**, 选择合适的单位。
```

3. 点击右上角 **Apply**。

添加 Counter 指标面板

本节创建一个 **Time Series** 面板，展示 Counter 类型指标的速率。Counter 的 `value` 是单调递增的累计值，需要使用通用 rate 模板计算速率。

1. 在 Dashboard 中，点击 **Add** > **Visualization**。
2. 选择数据源，切换到 **Code** 模式。

将 rate 模板的占位符替换为实际值。以下示例计算网络接收字节的速率，按网卡设备拆分：

```
```SQL
SELECT
 t.timestamp AS time,
 t.device AS metric,
 CASE
 WHEN UNIX_TIMESTAMP(t.timestamp) > UNIX_TIMESTAMP(t.prev_ts)
 AND t.value >= t.prev_value
 THEN (t.value - t.prev_value) / (UNIX_TIMESTAMP(t.timestamp) -
UNIX_TIMESTAMP(t.prev_ts))
 ELSE NULL
 END AS value
FROM (
 SELECT timestamp, CAST(attributes['device'] AS VARCHAR) AS device, value,
 LAG(value) OVER (PARTITION BY CAST(attributes['device'] AS VARCHAR) ORDER BY
timestamp) AS prev_value,
 LAG(timestamp) OVER (PARTITION BY CAST(attributes['device'] AS VARCHAR) ORDER
BY timestamp) AS prev_ts
 FROM otel.otel_metrics_sum
 WHERE metric_name = 'node_network_receive_bytes_total'
 AND $__timeFilter(timestamp)
) t
WHERE t.prev_ts IS NOT NULL
ORDER BY time
```



要替换为其他 Counter 指标，修改以下两处：

- metric\_name：改为目标指标名。
- PARTITION BY 和 SELECT 中的维度字段：改为该指标的拆分维度（如 attributes['cpu']、attributes['gc\_name'] 等）。
- 粘贴 SQL，将 **Format** 设为 **Time series**。
- 在 **Standard options > Unit** 中，选择合适的单位。
- 点击 **Apply**。

## 添加 Stat 面板

Stat 面板用于展示单个数值，如当前使用率或最新计数。SQL 返回一个数值即可。

1. 点击 **Add > Visualization**，切换到 **Code** 模式。

以下查询获取时间范围内最新的一条指标值：

```
SELECT value
FROM otel.otel_metrics_gauge
WHERE metric_name = 'node_memory_MemAvailable_bytes'
AND $__timeFilter(timestamp)
ORDER BY timestamp DESC
LIMIT 1
```

如果需要对多个指标做运算（如计算比率），参考优化面板中的「在单条 SQL 中查询多个指标」。

2. 粘贴 SQL，将 **Format** 设为 **Table**（Stat 面板使用 Table 格式）。
3. 在右侧面板设置中，将面板类型切换为 **Stat**。
4. 在 **Standard options > Unit** 中，选择合适的单位。
5. 点击 **Apply**。

## 添加 Table 面板

Table 面板适合展示多行多列数据，SQL 中的每个列别名作为表头。

1. 点击 **Add > Visualization**，切换到 **Code** 模式。

以下查询展示最近的日志明细：

```
SELECT
timestamp,
service_name,
```

```
severity_text,
body,
CAST(log_attributes['your_key'] AS VARCHAR) AS your_key
FROM otel.otel_logs
WHERE $__timeFilter(timestamp)
ORDER BY timestamp DESC
LIMIT 100
```

将 your\_key 替换为实际的日志属性字段名。要查看可用的属性字段，运行以下查询：

```
SELECT log_attributes FROM otel.otel_logs LIMIT 1;
```

2. 粘贴 SQL，将 **Format** 设为 **Table**。
3. 点击 **Apply**。

### 添加模板变量

模板变量为 Dashboard 添加交互式下拉筛选器，使用户无需修改 SQL 即可过滤数据。

#### 创建单选变量：

1. 点击 Dashboard 右上角齿轮图标，进入 **Settings > Variables > New variable**。
2. 按以下配置填写：

- **Name**: service\_name

- **Type**: Query

- **Data source**: 选择你的 Doris 数据源

- **Query**:

```
sql SELECT DISTINCT service_name FROM otel.otel_metrics_gauge WHERE
service_name != "" AND service_name IS NOT NULL AND $__timeFilter(timestamp)
ORDER BY service_name
```

1. 点击 **Apply**。

在面板 SQL 中使用 \$variable 语法引用单选变量：

```
AND service_name = '$service_name'
```

#### 创建多选变量：

1. 新建变量，按以下配置填写：

- **Name:** namespace
- **Type:** Query
- **Multi-value:** 勾选
- **Include All option:** 勾选
- **Query:**

```
sql SELECT DISTINCT CAST(resource_attributes['k8s.namespace.name'] AS VARCHAR)
AS __text FROM otel.otel_metrics_gauge WHERE metric_name = 'k8s.pod.phase'
AND timestamp >= NOW() - INTERVAL 1 HOUR ORDER BY 1
```

1. 点击 **Apply**。

Note: 列别名 \_\_text 是 Grafana 约定，用于控制下拉框中的显示文本。

在面板 SQL 中使用 `${variable:sqlstring}` 语法配合 `IN()` 引用多选变量：

```
AND CAST(resource_attributes['k8s.namespace.name'] AS VARCHAR) IN ($
{namespace:sqlstring})
```

Caution: 多选变量如果不使用 `:sqlstring`，生成的 SQL 会有语法错误。

### 创建级联变量：

变量的可选值可以依赖其他变量。例如，以下 Pod 变量根据已选的 Namespace 过滤可选值：

```
SELECT DISTINCT CAST(resource_attributes['k8s.pod.name'] AS VARCHAR) AS __text
FROM otel.otel_metrics_gauge
WHERE metric_name = 'k8s.pod.phase'
AND timestamp >= NOW() - INTERVAL 1 HOUR
AND CAST(resource_attributes['k8s.namespace.name'] AS VARCHAR) IN ($
{namespace:sqlstring})
ORDER BY 1
```

### 优化面板

以下是预置 Dashboard 中使用的常见优化技巧。

**调整时间分桶间隔。** 根据时间范围选择合适的分桶大小。修改 `FLOOR(UNIX_TIMESTAMP(timestamp) / N) * N` 中的 N 值：

- 20 秒：适合短时间实时监控。
- 60 秒：适合小时级概览。
- 300 秒：适合天级趋势。

**设置有意义的线条名称。** 使用 CONCAT() 组合多个字段：

```
CONCAT(device, ' read') AS metric
```

使用 CASE WHEN 将数值映射为可读文本：

```
CASE WHEN value = 2 THEN 'Running'
 WHEN value = 1 THEN 'Pending'
 WHEN value = 3 THEN 'Succeeded'
 WHEN value = 4 THEN 'Failed'
 ELSE 'Unknown'
END AS status
```

**处理除零和空值。** 使用 NULLIF 防止除零，使用 COALESCE 提供默认值：

```
/ NULLIF(SUM(...), 0)
COALESCE(restarts, 0)
```

**在单条 SQL 中查询多个指标。** 用 CASE WHEN metric\_name 替代 JOIN：

```
SELECT timestamp AS time,
 SUM(CASE WHEN metric_name = 'node_memory_MemAvailable_bytes' THEN value END)
 AS available,
 SUM(CASE WHEN metric_name = 'node_memory_MemTotal_bytes' THEN value END) AS
 total
FROM otel.otel_metrics_gauge
WHERE metric_name IN ('node_memory_MemTotal_bytes',
 'node_memory_MemAvailable_bytes')
GROUP BY timestamp
```

**过滤噪声数据。** 排除虚拟网卡：

```
AND CAST(attributes['device'] AS VARCHAR) NOT LIKE 'veth%'
AND CAST(attributes['device'] AS VARCHAR) NOT LIKE 'br-%'
AND CAST(attributes['device'] AS VARCHAR) != 'lo'
```

只保留真实文件系统：

```
AND CAST(attributes['fstype'] AS VARCHAR) IN ('ext4', 'xfs', 'btrfs')
```

## 参考

### OTel 数据表结构

#### Metrics 表 (gauge / sum / histogram 共有字段)

- 字段: service\_name; 类型: varchar(200); 说明: 服务名称
- 字段: timestamp; 类型: datetime(6); 说明: 数据时间戳
- 字段: service\_instance\_id; 类型: varchar(200); 说明: 服务实例标识
- 字段: metric\_name; 类型: varchar(200); 说明: 指标名称
- 字段: metric\_description; 类型: text; 说明: 指标描述
- 字段: metric\_unit; 类型: text; 说明: 指标单位
- 字段: attributes; 类型: variant (JSON); 说明: 指标维度属性
- 字段: resource\_attributes; 类型: variant (JSON); 说明: 资源属性
- 字段: scope\_name; 类型: text; 说明: 采集器名称

gauge 独有字段: value (double)

sum 额外字段: value (double)、aggregation\_temporality (text)、is\_monotonic (boolean)

histogram 额外字段: count (bigint)、sum (double)、bucket\_counts (array)、explicit\_bounds (array)、min (double)、max (double)

#### Logs 表

- 字段: timestamp; 类型: datetime(6); 说明: 日志时间戳
- 字段: service\_name; 类型: varchar(200); 说明: 服务名称
- 字段: service\_instance\_id; 类型: varchar(200); 说明: 服务实例标识
- 字段: trace\_id; 类型: varchar(200); 说明: 关联 Trace ID
- 字段: span\_id; 类型: text; 说明: 关联 Span ID
- 字段: severity\_number; 类型: int; 说明: 日志级别编号
- 字段: severity\_text; 类型: text; 说明: 日志级别 (INFO / WARN / ERROR)
- 字段: body; 类型: text; 说明: 日志正文
- 字段: resource\_attributes; 类型: variant (JSON); 说明: 资源属性
- 字段: log\_attributes; 类型: variant (JSON); 说明: 日志属性

#### Traces 表

- 字段: timestamp; 类型: datetime(6); 说明: Span 开始时间

- 字段: service\_name; 类型: varchar(200); 说明: 服务名称
- 字段: trace\_id; 类型: varchar(200); 说明: Trace ID
- 字段: span\_id; 类型: text; 说明: Span ID
- 字段: parent\_span\_id; 类型: text; 说明: 父 Span ID
- 字段: span\_name; 类型: text; 说明: Span 名称
- 字段: span\_kind; 类型: text; 说明: Span 类型 (CLIENT / SERVER / INTERNAL)
- 字段: end\_time; 类型: datetime(6); 说明: Span 结束时间
- 字段: duration; 类型: bigint; 说明: 耗时 (纳秒)
- 字段: span\_attributes; 类型: variant (JSON); 说明: Span 属性
- 字段: events; 类型: array; 说明: Span 事件
- 字段: links; 类型: array; 说明: Span 关联
- 字段: status\_code; 类型: text; 说明: 状态码 (OK / ERROR / UNSET)
- 字段: status\_message; 类型: text; 说明: 状态消息
- 字段: resource\_attributes; 类型: variant (JSON); 说明: 资源属性

## 语法速查

- 用途: 时间过滤 (宏) ; 写法: \$\_\_timeFilter(timestamp)
- 用途: 时间过滤 (手动) ; 写法: timestamp >= FROM\_UNIXTIME(\$\_\_from/1000)
- 用途: 时间分桶 (20 秒) ; 写法: FLOOR(UNIX\_TIMESTAMP(timestamp) / 20) \* 20 AS time
- 用途: 时间分桶 (1 分钟) ; 写法: UNIX\_TIMESTAMP(DATE\_FORMAT(timestamp, '%Y-%m-%d %H:%i:00')) \* 1000 AS time
- 用途: 属性访问; 写法: attributes['key']
- 用途: 属性 CAST; 写法: CAST(attributes['key'] AS VARCHAR)
- 用途: 单选变量; 写法: service\_name = '\$service\_name'
- 用途: 多选变量; 写法: IN (\${namespace:sqlstring})
- 用途: 防除零; 写法: / NULLIF(..., 0)
- 用途: 默认值; 写法: COALESCE(..., 0)
- 用途: 线条命名; 写法: CONCAT(device, ' read') AS metric
- 用途: 状态映射; 写法: CASE WHEN value = 2 THEN 'Running' ... END
- 用途: URL 去参数; 写法: SUBSTRING\_INDEX(url, '?', 1)
- 用途: 过滤虚拟网卡; 写法: NOT LIKE 'veth%' + NOT LIKE 'br-%' + != 'lo'